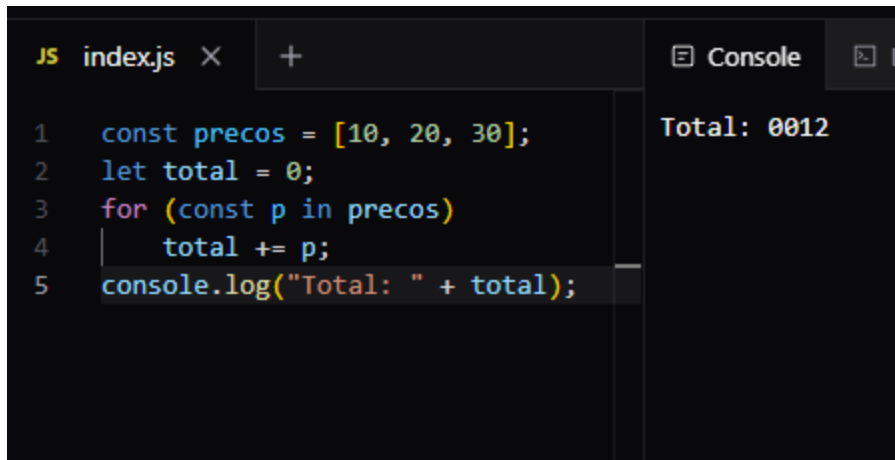


Resolução de Bugs

Este documento reúne a análise e correção dos 6 exercícios de código apresentados, abordando JavaScript, Python, Java, C e Go.

1. JavaScript

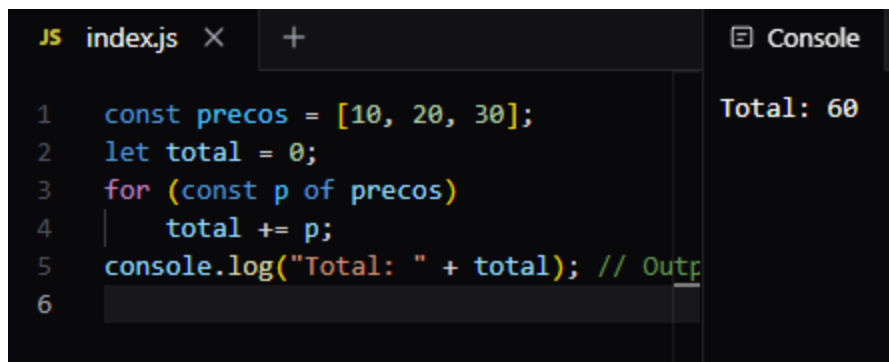
- (a) Saída: Total: 0012



```
JS index.js x + Console
1 const precos = [10, 20, 30];
2 let total = 0;
3 for (const p in precos)
4   total += p;
5 console.log("Total: " + total);
```

Total: 0012

-
- (b) Regra que causa o problema: O laço for...in itera sobre as **chaves/índices** do array (que são *strings*: "0", "1", "2"), e não sobre os valores. Ao somar 0 + "0" + "1" + "2", ocorre a concatenação de strings em vez de soma numérica.
- (c) Como corrigir: Utilize o laço for...of para iterar diretamente sobre os valores do array:



```
JS index.js x + Console
1 const precos = [10, 20, 30];
2 let total = 0;
3 for (const p of precos)
4   total += p;
5 console.log("Total: " + total); // Output
6
```

Total: 60

2. Python

- (a) Saída: 180.0

```
main.py x + Console
1 def desconto(preco, d=None):
2     d = d or 10 # padrão: 10%
3     return preco * (100 - d) / 100
4
5 print(desconto(200, 0))
```

180.0

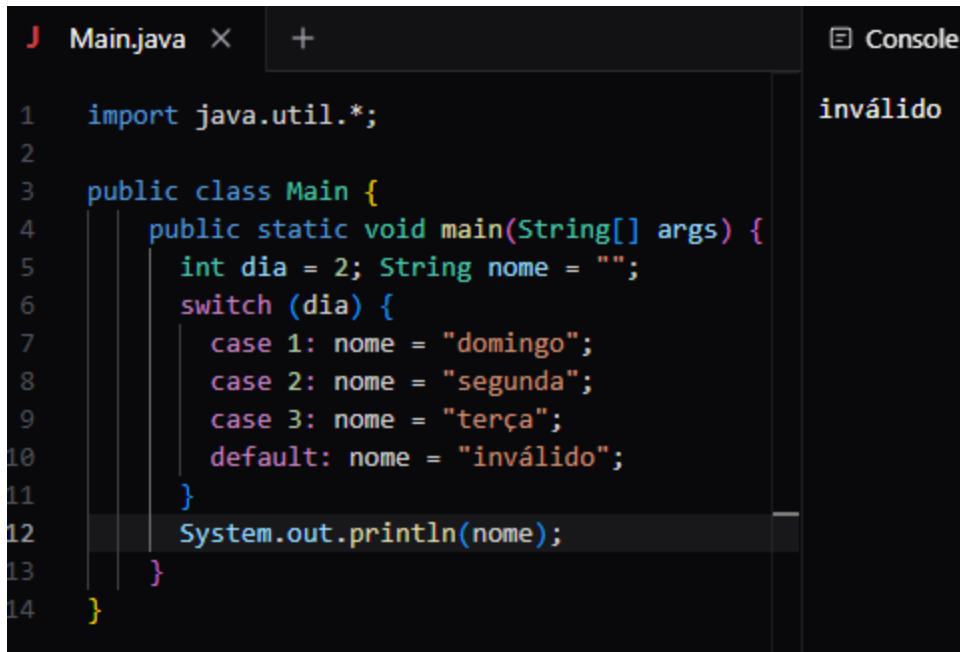
-
- **(b) Regra que causa o problema:** A expressão `d = d or 10` avalia o valor de `d` em contexto booleano. Em Python, o número 0 é tratado como **falso** (False). Portanto, quando `d=0` é passado, o operador `or` ignora o 0 e atribui o valor padrão 10.
- **(c) Como corrigir:** Verifique explicitamente se `d` é None usando o operador `is`:

```
main.py x + Console
1 def desconto(preco, d=None):
2     if d is None:
3         d = 10 # padrão: 10%
4     return preco * (100 - d) / 100
5
6 print(desconto(200, 0)) # Output: 200.0
```

200.0

3. Java

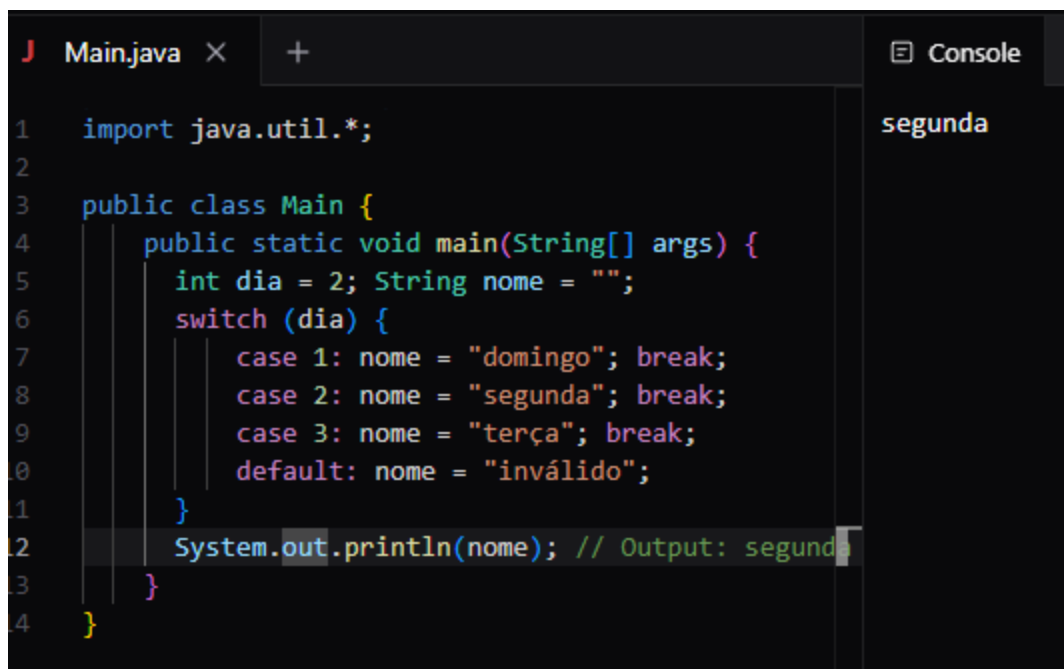
- **(a) Saída:** inválido



```
1  import java.util.*;
2
3  public class Main {
4      public static void main(String[] args) {
5          int dia = 2; String nome = "";
6          switch (dia) {
7              case 1: nome = "domingo";
8              case 2: nome = "segunda";
9              case 3: nome = "terça";
10             default: nome = "inválido";
11         }
12         System.out.println(nome);
13     }
14 }
```

Console: inválido

-
- **(b) Regra que causa o problema:** A ausência da instrução break ao final de cada case causa o efeito de *fall-through* (queda). O código encontra o case 2, define nome = "segunda", mas continua executando os blocos seguintes (case 3 e default) até o final do switch, sobrescrevendo o valor final com "inválido".
- **(c) Como corrigir:** Adicione a instrução break ao final de cada bloco do switch para interromper a execução:

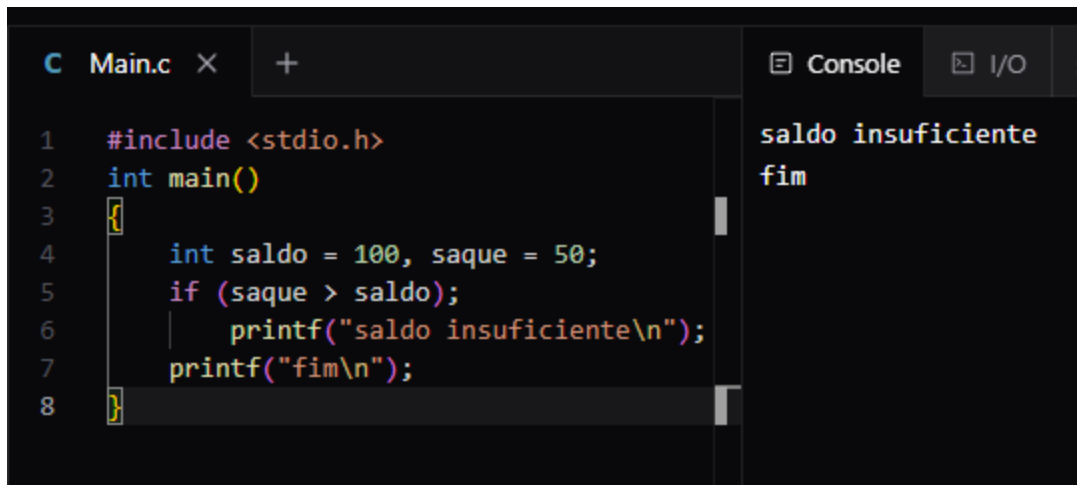


```
1  import java.util.*;
2
3  public class Main {
4      public static void main(String[] args) {
5          int dia = 2; String nome = "";
6          switch (dia) {
7              case 1: nome = "domingo"; break;
8              case 2: nome = "segunda"; break;
9              case 3: nome = "terça"; break;
10             default: nome = "inválido";
11         }
12         System.out.println(nome); // Output: segunda
13     }
14 }
```

Console: segunda

4. C

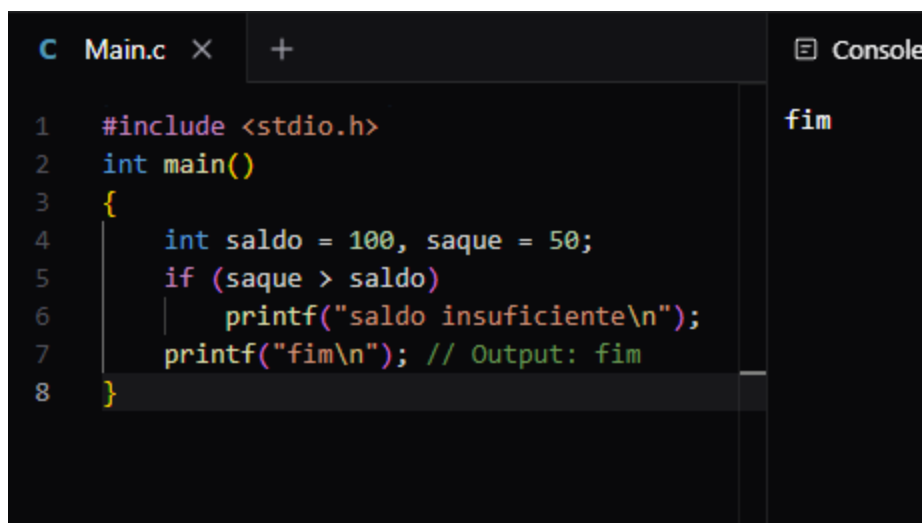
- **(a) Saída:**
saldo insuficiente
fim



```
1  #include <stdio.h>
2  int main()
3  {
4      int saldo = 100, saque = 50;
5      if (saque > saldo);
6          printf("saldo insuficiente\n");
7          printf("fim\n");
8  }
```

Console output: saldo insuficiente
fim

- **(b) Regra que causa o problema:** O ponto e vírgula ; logo após a condição if (saque > saldo); encerra imediatamente a estrutura do if como uma instrução vazia. Por conta disso, o bloco do printf("saldo insuficiente\n"); deixa de estar condicionado ao if e passa a ser executado incondicionalmente.
- **(c) Como corrigir:** Remova o ponto e vírgula após a instrução if:



```
1  #include <stdio.h>
2  int main()
3  {
4      int saldo = 100, saque = 50;
5      if (saque > saldo)
6          printf("saldo insuficiente\n");
7          printf("fim\n"); // Output: fim
8  }
```

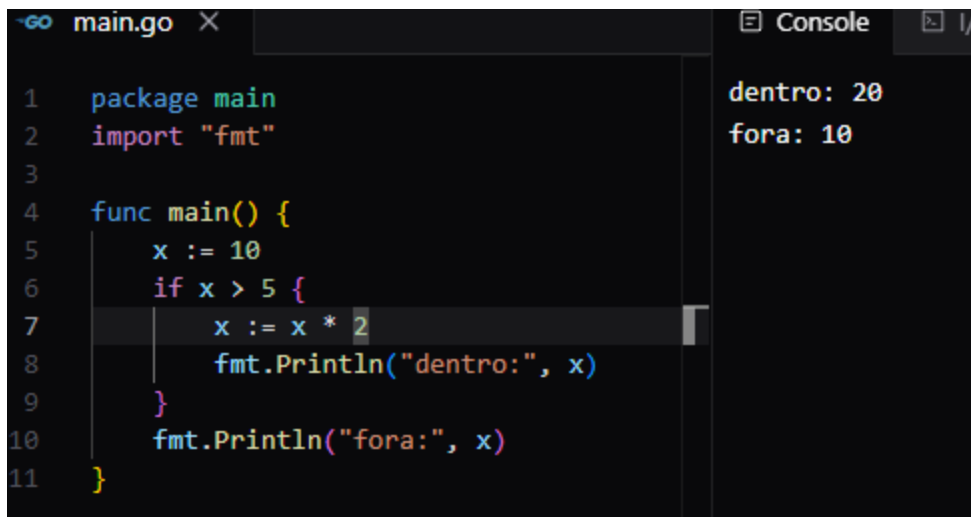
Console output: fim

5. Go

- (a) Saída:

dentro: 20

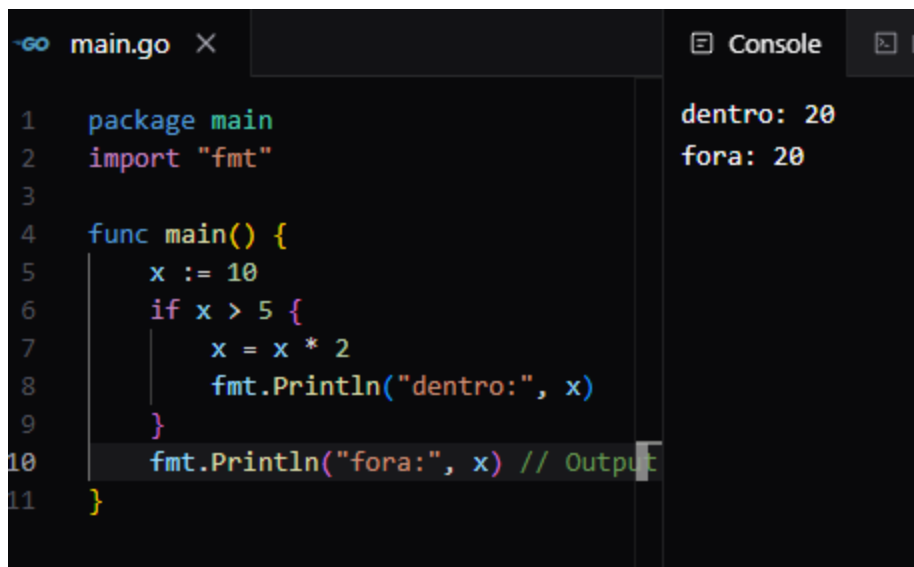
fora: 10



```
1 package main
2 import "fmt"
3
4 func main() {
5     x := 10
6     if x > 5 {
7         x := x * 2
8         fmt.Println("dentro:", x)
9     }
10    fmt.Println("fora:", x)
11 }
```

dentro: 20
fora: 10

- (b) Regra que causa o problema: O uso do operador de declaração curta := dentro do bloco if cria uma **nova variável local** chamada x (sombreamento de variável ou *variable shadowing*), em vez de reatribuir valor à variável x do escopo externo.
- (c) Como corrigir: Utilize o operador de atribuição simples = dentro do if para alterar o valor da variável externa existente:



```
1 package main
2 import "fmt"
3
4 func main() {
5     x := 10
6     if x > 5 {
7         x = x * 2
8         fmt.Println("dentro:", x)
9     }
10    fmt.Println("fora:", x) // Output
11 }
```

dentro: 20
fora: 20

6. Java

- (a) Saída: 0.0%

```
1 import java.util.*;
2
3 public class Main {
4     public static void main(String[] args) {
5         int acertos = 7, total = 10;
6         double taxa = acertos / total * 100;
7         System.out.println(taxa + "%");
8     }
9 }
```

Console: 0.0%

-
- **(b) Regra que causa o problema:** Como acertos e total são do tipo inteiro (int), a operação `acertos / total` realiza uma **divisão inteira**. O resultado de `7 / 10` é 0 (a parte decimal é descartada) e, em seguida, `0 * 100` resulta em 0, que só então é convertido para double (0.0).
- **(c) Como corrigir:** Faça o *casting* de pelo menos uma das variáveis para double antes de realizar a divisão:

```
1 import java.util.*;
2
3 public class Main {
4     public static void main(String[] args) {
5         int acertos = 7, total = 10;
6         double taxa = (double) acertos / total * 100;
7         System.out.println(taxa + "%"); // Output: 70.0%
8     }
9 }
```

Console: 70.0%